

Pekaren `this`

Varje objekt kan referera till sig själv ifall det är nödvändigt. Ibland kanske en objekt vill skicka en pekare till sig själv till någon extern funktion eller metod. Man använder då en speciell pekare som heter `this`. Denna är alltid definierad då man är *inne i* en metod i en klass, i övriga fall är den odefinierad. Man kan alltså inte accessera `this` från t.ex. funktionen `main()`, utan man måste vara inne i en metod. En statisk metod (se [avsnittet Statiska metoder och medlemmar](#)) kan ej heller användas, eftersom man då inte manipulerar ett objekt. Pekaren `this` är alltid en pekare till samma datatyp som klassen man just då "är inne i".

Man kan tänka sig ett osynligt `this` framför alla accesser till medlemmar och anrop av metoder inne i en klass. Om vi t.ex. har vår klass `Coordinate` som normalt ser ut på följande sätt:

```
class Coordinate {
public:
    // konstruktor
    Coordinate (float X, float Y) { m_X = X; m_Y = Y; };

    // accessera medlemmar
    float x () { return m_X; };
    float y () { return m_Y; };
    void setX (float X) { m_X = X; };
    void setY (float Y) { m_Y = Y; };

private:
    // medlemmar
    float m_X;
    float m_Y;
};
```

kan vi tänka oss att metoderna är definierade på följande sätt:

```
...
Coordinate (float X, float Y) { this->m_X = X; this->m_Y = Y; };

// accessera medlemmar
float x () { return this->m_X; };
float y () { return this->m_Y; };
void setX (float X) { this->m_X = X; };
void setY (float Y) { this->m_Y = Y; };
...
```

Det blir jobbigt att alltid skriva ut `this` på alla ställen så man kan (och gör i de flesta fallen) lämna bort pekaren. Det finns vissa fall då man vill använda sig av pekaren `this`, t.ex. då man vill att en metod skall returnera objektet självt som returvärde.